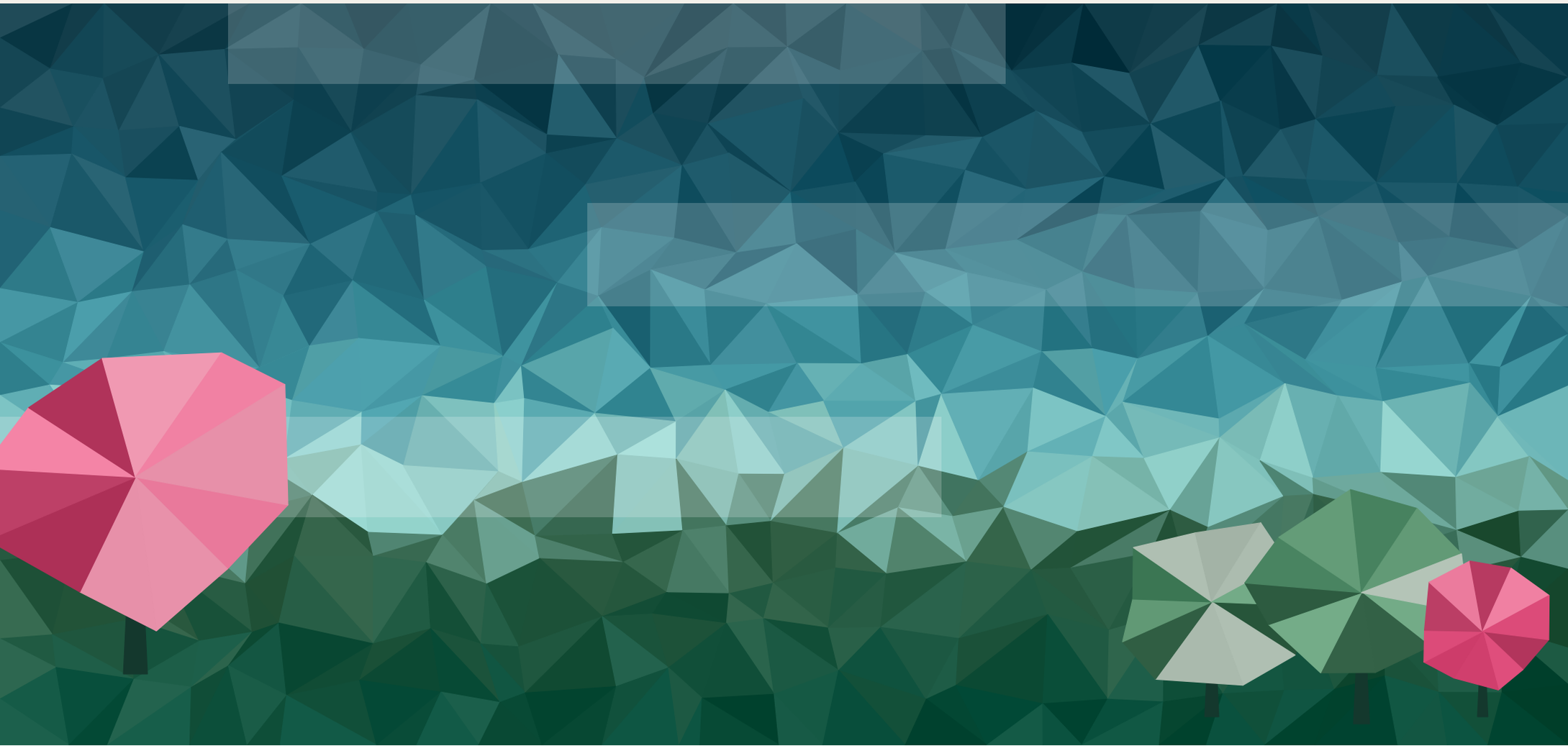
ECOCIENTE · POSTGRESQL

BACKUP E RECUPERAÇÃO

EcoCiente md-populador-pgsql gerado em 02/11/2026 scripts/backup/ testado em PostgreSQL 16



O RESUMO EM CINCO NÚMEROS

Metas definidas para o projeto e os tempos de um teste real. Cada etiqueta diz qual é qual.

RPO

24 h

perda máxima de dados

No pior caso perde-se o que foi escrito desde o último dump diário.

PREMISSA

RTO

1 h

meta para voltar a operar

Criar o serviço, restaurar, conferir e trocar ECOCIENTE_DSN. A restauração em si foi medida em segundos.

PREMISSA

FREQUÊNCIA

DIÁRIA

dump completo

pg_dump -Fc todo dia, e --schema-only a cada mudança de schema.

PREMISSA

RETENÇÃO

7 + 4

diários + semanais

Configurável por ECOCIENTE_RETENCAO_DIARIOS e ECOCIENTE_RETENCAO_SEMANAIS.

PREMISSA

TEMPOS MEDIDOS

0,5 s

2,0 s

backup · restauração

Escala leve: banco de 28,1 MB, dump de 1,5 MB.

MEDIDO

O QUE ESTE DOCUMENTO NÃO AFIRMA

Três limites que valem para todo o documento.

- Que os tempos valham para produção: foram medidos na escala leve, num PostgreSQL local em Docker, sem rede. Contra a Aiven entram latência e o tamanho real.
- Que RPO, RTO, frequência e retenção sejam SLA do provedor: são metas definidas aqui.
- O que o provedor gerenciado oferece (backups automáticos, PITR) depende do plano contratado. Confira no console antes de contar com isso.

Estratégia

Dump lógico diário, guardado fora do banco, mais o que o provedor já faz.

O QUE É FEITO

O quê	Comando	Quando	Onde fica
Banco inteiro	`pg_dump -Fc`	diário	backups/ecociente_AAAAMMDD_HHMMSS.dump
Só o schema	`pg_dump --schema-only`	a cada mudança em sql/	….schema.sql, com o mesmo carimbo
Retenção	`backup.sh / backup.ps1`	depois de cada backup bem-sucedido	7 diários + 4 semanais
Cópia externa	`cópia do arquivo`	diária	fora do servidor do banco: outra máquina ou armazenamento em nuvem da equipe; nunca só no mesmo serviço do banco

POR QUE MANTER O BACKUP LÓGICO

Aiven e Neon fazem backups automáticos e oferecem restauração a um ponto no tempo (PITR), conforme o plano. Mantemos o backup lógico mesmo assim, por quatro motivos.

1. **Independe da conta.** Os backups do provedor em geral somem junto com o serviço; o dump fica com a equipe.
2. **É portátil.** Serve entre provedores e versões: Aiven, Neon ou PostgreSQL local.
3. **Restaura uma tabela só,** sem voltar o serviço inteiro no tempo.
4. **Dá para testar todo mês** num PostgreSQL local, sem tocar na produção.

Como fazer backup

Um comando, do `backup.sh` ou do `backup.ps1`.

1. Defina `ECOCIENTE_DSN` no ambiente ou no `.env` (o script lê só essa linha; veja `.env.example`).
2. Tenha o `pg_dump` 16 ou mais novo no PATH, ou aponte `PG_BIN` para a pasta dele. A versão deve ser igual ou maior que a do servidor.
3. Rode o script (comandos abaixo), da raiz do repositório.
4. Confira a linha `ok:` e as linhas `mantido:` e `apagado:` da retenção.
5. Copie o .dump para fora do servidor do banco.

LINUX, MACOS E GIT BASH

```
scripts/backup/backup.sh
```

WINDOWS (POWERSHELL)

```
.\scripts\backup\backup.ps1
```

Se o pg_dump falhar, o script imprime ERRO: pg_dump falhou (código N). Nenhum backup novo foi gravado. e sai com código 1. Não fica arquivo parcial.

VARIÁVEIS DE AMBIENTE

Variável	Padrão	Para quê
`ECOCIENTE_DSN`	`(obrigatória)`	conexão com o banco; também lida da linha ECOCIENTE_DSN= do .env
`ECOCIENTE_BACKUP_DIR`	`<raiz>/backups`	pasta onde os arquivos são gravados
`ECOCIENTE_RETENCAO_DIARIOS`	`7`	quantos dumps mais novos manter; deve ser 1 ou mais
`ECOCIENTE_RETENCAO_SEMANAIS`	`4`	quantas semanas anteriores manter, com o dump mais novo de cada uma; 0 desliga
`PG_BIN`	`(PATH)`	pasta do pg_dump, se não estiver no PATH

AGENDAMENTO DIÁRIO

CRON (LINUX)

```
0 2 * * * cd /caminho/md-populador-pgsql && ECOCIENTE_DSN=... scripts/backup/backup.sh
```

AGENDADOR DE TAREFAS (WINDOWS)

```
schtasks /Create /SC DAILY /ST 02:00 /TN "EcoCiente backup" ^
  /TR "powershell -NoProfile -File C:\caminho\md-populador-pgsql\scripts\backup\backup.ps1"
```

Como restaurar

Banco inteiro, uma tabela ou só a conferência por contagem.

BANCO INTEIRO, NUM BANCO NOVO

1. Crie um banco novo e vazio (no serviço novo ou num PostgreSQL local).
2. Rode `restore.sh` com o dump e a DSN do banco novo. Ele executa `pg_restore --clean --if-exists --no-owner --no-privileges --exit-on-error`.
3. O script recusa um destino igual a `ECOCIENTE_DSN`. Para restaurar por cima de propósito, exporte `ECOCIENTE_ALLOW_RESET=1`.
4. Confira com `--comparar` (abaixo).

LINUX, MACOS E GIT BASH

```
scripts/backup/restore.sh backups/ecociente_AAAAMMDD_HHMMSS.dump \
  "postgres://usuario:senha@host:5432/ecociente_restaurado?sslmode=require"
```

WINDOWS (POWERSHELL)

```
.\scripts\backup\restore.ps1 backups\ecociente_AAAAMMDD_HHMMSS.dump `
  "postgres://usuario:senha@host:5432/ecociente_restaurado?sslmode=require"
```

UMA TABELA ISOLADA

Restaurar uma tabela só (pg_restore --data-only -t). Sem --truncar, linhas que já existem colidem com a chave primária. O --truncar não usa CASCADE: se outras tabelas referenciam a restaurada por FK, liste todas juntas com -t. O caminho mais seguro é restaurar o dump inteiro num banco novo e copiar as linhas de lá.

```
scripts/backup/restore.sh -t tb_cursos [--truncar] backups/ecociente_AAAAMMDD_HHMMSS.dump "<dsn_destino>"
.\scripts\backup\restore.ps1 backups\ecociente_AAAAMMDD_HHMMSS.dump "<dsn_destino>" -Tabela tb_cursos [-Truncar]
```

CONFERÊNCIA POR CONTAGEM

`--comparar` (no PowerShell, `-Comparar`) mostra origem × restaurado das 17 tabelas principais, marca `<-- DIVERGE` e sai com código 1 se algo diferir. Precisa de `ECOCIENTE_DSN` apontando para a origem, e a origem não pode ter mudado desde o backup. MEDIDO

CONTAGENS CONFERIDAS NO TESTE

Tabela	Origem	Restaurado
tb_usuarios	435	435
tb_condominios	7	7
tb_moradores	303	303
tb_cooperativas	3	3
tb_postagens	1595	1595
tb_rel_votos_postagens	8591	8591
tb_agendamentos_coletas	42	42
tb_visitas_coletas	155	155
tb_cursos	40	40
tb_aulas	238	238
tb_quizzes	20	20
tb_tentativas_quiz	134	134
tb_movimentacoes_pontos	1916	1916
tb_notificacoes	4190	4190
tb_log_auditoria	16020	16020
fato_postagens	1595	1595
fato_coletas	155	155
Contagens iguais: 17 de 17		

Recuperação de falhas

Seis falhas previstas, o que se vê e o que fazer.

FALHA, SINTOMA E PROCEDIMENTO

Falha	Sintoma	Procedimento
Carga do populador interrompida	python src/main.py para no meio (queda de conexão, timeout); as tabelas ficam com contagens parciais.	O populador trunca e recarrega: rode de novo com a mesma escala e --force (ou ECOCIENTE_ALLOW_RESET=1).
Schema corrompido	relation … does not exist, colunas faltando, ou a checagem de schema de python ui.py acusa diferença.	Banco novo; psql "$ECOCIENTE_DSN" -v ON_ERROR_STOP=1 -f sql/ecociente_schema.sql e depois -f sql/ecociente_datamart.sql, nessa ordem. Depois repopule ou restaure só os dados.
Data Mart inconsistente	fato_postagens e fato_coletas com números diferentes do OLTP; dashboard desatualizado.	CALL sp_carga_datamart(); (refresh completo).
Exclusão acidental de dados	linhas sumiram de uma tabela.	Restauração da tabela a partir do último dump (página "Como restaurar").
Falha na migração RPA	a migração parou no meio ou deixou registros pela metade.	Carga transacional, tb_migracao_ids_map (mapa legado para novo, que torna a nova execução idempotente) e reconciliação do repo md-rpa-integration.
Perda total do servidor	serviço inacessível ou excluído.	Restauração completa a partir do dump externo num serviço novo, conferência por contagem e troca de ECOCIENTE_DSN.

Nota: num banco vazio o ecociente_schema.sql atual falha com relation "tb_recuperacoes_senha" does not exist, porque o índice idx_esqueci_senha_usuario_id vem antes da tabela. Enquanto o arquivo não for corrigido no repo, mova esse CREATE INDEX para depois do CREATE TABLE tb_recuperacoes_senha numa cópia.

Teste de restauração mensal

Todo mês, na primeira semana, pelo responsável pelo banco.

Quem executa: o responsável pelo banco (perfil DBA / Admin do catálogo de dados), com uma segunda pessoa conferindo o resultado.

1. O último dump existe e tem menos de 24 h.
2. O tamanho é coerente com o do mês anterior.
3. Suba um PostgreSQL 16 local: `docker run --rm -d -p 15432:5432 -e POSTGRES_PASSWORD=teste postgres:16`.
4. Crie um banco novo e vazio nele.
5. Faça um backup novo e rode `restore.sh --comparar` logo em seguida, para a origem não ter mudado.
6. Confira que as contagens são iguais (sem linha DIVERGE).
7. Aplique o `.schema.sql` num banco vazio e confira que não dá erro.
8. Anote os tempos de backup e de restauração e compare com o mês anterior.
9. Apague o banco de teste e confira que a retenção mantém 7 + 4 arquivos.

REGISTRO

Data	Responsável	Tempo de restauração	Contagens ok?

COMO MEDIMOS

Um teste real, num PostgreSQL 16 local.

COMO O TESTE FOI FEITO

1. Contêiner postgres:16 local.
2. Schema (com a correção numa cópia, ver a nota da página de falhas) e depois o Data Mart.
3. Populador com --escala leve --force: 17,4 s.
4. CALL sp_carga_datamart();
5. backup.sh dentro do contêiner: 0,5 s, dump de 1,5 MB.
6. restore.sh --comparar num banco novo (ecociente_restaurado): 2,0 s, 17 de 17 contagens iguais.

LIMITAÇÕES

- Escala leve, uma máquina local, uma execução, sem rede.
- O banco tem 28,1 MB; o de produção será maior e os tempos crescem com ele.
- O tempo de restauração é só a etapa do pg_restore; o script inteiro, com as contagens, leva mais.
- Os scripts .ps1 foram validados de ponta a ponta no PowerShell 7 (em contêiner); no Windows PowerShell 5.1 só até a checagem do pg_dump.

ONDE ESTÁ CADA COISA

Tudo sai do repositório: scripts, schema e este documento.

ARQUIVOS

- `scripts/backup/backup.sh` e backup.psl: geram o dump e o .schema.sql
- `scripts/backup/restore.sh` e restore.psl: restauram e conferem
- `backups/` pasta dos dumps (ignorada pelo git)
- `sql/ecociente_schema.sql` e ecociente_datamart.sql: o schema, nessa ordem
- `documentos/catalogo_dados.pdf` o catálogo de dados
- `python documentos/gerar_documentos.py` regera este documento

